Funções definidas pelo usuário em Python

1. Função para calcular o dobro

Defina uma função **dobro(n)** que receba um número e retorne seu dobro.

2. Função para calcular a média

Defina uma função **media(a, b, c)** que receba três números como parâmetros posicionais e retorne a média aritmética.

3. Função para verificar paridade

Crie uma função **eh_par(n)** que retorne **True** se n for par e **False** caso contrário.

4. Conversão de temperatura

Crie uma função **celsius_para_fahrenheit(c)** que converta uma temperatura Celsius para Fahrenheit. Use: $F = 59C + 32$.

5. Área de um retângulo

Crie uma função **area_retangulo(base, altura)** que calcule e retorne a área de um retângulo.

6. Parâmetro opcional

Crie uma função **saudacao(nome, mensagem="Olá")**. Se apenas o nome for fornecido, deve imprimir:

```
Olá, João
```

Se uma mensagem for fornecida, deve utilizá-la:

```
Bom dia, João
```

7. Potência com expoente padrão

Crie uma função **potencia(base, expoente=2)**. O expoente deverá assumir o valor 2 quando não for informado.

Exemplo:

```
potencia(5)      # 25
potencia(5, 3)   # 125
```

8. Argumentos posicionais e nomeados

Crie uma função:

```
def apresentar(nome, idade, cidade):
  ...
```

Faça chamadas utilizando:

a) somente argumentos posicionais;
b) somente argumentos nomeados;
c) uma combinação de argumentos posicionais e nomeados.

9. Valor padrão para desconto

Crie uma função **preco_final(preco, desconto=0)** que retorne o preço depois de aplicar o desconto percentual.

Exemplos:

```
preco_final(100)      # 100
preco_final(100, 10)  # 90
```

10. Função com vários parâmetros opcionais

Crie uma função:

produto(nome, preco, quantidade=1, desconto=0)

que calcule o valor total da compra:

total = preco × quantidade × (1−desconto/100)

11. Modificação de um inteiro

Observe o código:

```
def modificar(x):
  x = x + 10

numero = 20
modificar(numero)
print(numero)
```

Determine a saída e explique por que **numero** não foi alterado. Depois, escreva uma versão que retorne o novo valor.

12. Modificação de uma string

Crie uma função maiusculas(texto) que receba uma string e retorne uma nova string em letras maiúsculas. Verifique se a string original foi modificada.

13. Modificação de uma lista

Crie uma função **adicionar(lista, valor)** que acrescente **valor** ao final da lista.

Teste:

```
numeros = [1, 2, 3]
adicionar(numeros, 4)
print(numeros)
```

Explique por que a lista original foi modificada.

14. Lista e atribuição

Considere:

```
def modificar(lista):
    lista = lista + [10]

numeros = [1, 2, 3]
modificar(numeros)
print(numeros)
```

Determine a saída e explique a diferença entre esse exercício e o anterior.

15. Lista com append

Crie uma função duplicar_elementos(lista) que modifique a lista recebida, duplicando cada elemento.

Exemplo:

```
lista = [1, 2, 3]
duplicar_elementos(lista)
```

deve produzir:

```
[2, 4, 6]
```

16. Modificação de um dicionário

Crie uma função **atualizar_aluno(aluno, idade)** que altere a idade armazenada no dicionário.

Por exemplo:

```
aluno = {"nome": "Ana", "idade": 20}
```

Após a chamada:

```
atualizar_aluno(aluno, 21)
```

o dicionário deverá ser:

```
{'nome': 'Ana', 'idade': 21}
```

17. Dicionário versus atribuição

Analise:

```
def modificar(pessoa):
    pessoa = {"nome": "Carlos"}

dados = {"nome": "Ana"}
modificar(dados)
print(dados)
```

Explique por que dados continua contendo {"nome": "Ana"}.

18. Função que modifica um conjunto

Crie uma função **adicionar_elementos(conjunto, elementos)** que acrescente todos os elementos de uma lista ao conjunto.

Exemplo:

```
numeros = {1, 2, 3}
adicionar_elementos(numeros, [3, 4, 5])
```

Resultado:

```
{1, 2, 3, 4, 5}
```

19. Função que retorna uma nova lista

Crie uma função **pares(lista)** que receba uma lista de números e não modifique a lista original. A função deverá retornar uma nova lista contendo apenas os números pares.

Exemplo:

```
numeros = [1, 2, 3, 4, 5, 6]
resultado = pares(numeros)
print(resultado)
print(numeros)
```

Resultado:

```
[2, 4, 6]
[1, 2, 3, 4, 5, 6]
```

20. Desafio — cadastro de alunos

Crie uma função:

cadastrar_aluno(alunos, nome, idade, curso="Computação")

que receba um dicionário **alunos** e acrescente um novo aluno. O dicionário deverá utilizar o nome do aluno como chave e armazenar idade e curso.

Por exemplo:

```
alunos = {}
cadastrar_aluno(alunos, "Ana", 20)
cadastrar_aluno(alunos, "Carlos", 22, "Matemática")
```

O resultado esperado é:

```
{
  "Ana": {"idade": 20, "curso": "Computação"},
  "Carlos": {"idade": 22, "curso": "Matemática"}
}
```